Name of Student : Raghav Mahesh Sarda	**PRN:** 25070122***235***
Subject: Programming with Java	**Semester: III**
ASSIGNMENT No : 7	**Year: AY 2026-27**
TITLE : Inheritance and Interfaces	

Problem Statement

Implement a Java program to demonstrate inheritance and interfaces using suitable real-world examples.

Learning Objectives

To implement inheritance and interfaces for achieving code reusability and abstraction.

Program:

ShapeApplication.java

```
// Assignment 7 - Exercise 1: Shape application using Inheritance and
Interfaces
import java.util.*;
interface Drawable // Interface demonstration for abstraction
{
   void draw(); // Abstract method to be implemented by child classes
}

class Shape // Parent class demonstrating inheritance
{
   String color;
   Shape(String c) // Parent constructor
   {
       color=c;
   }
}

class Circle extends Shape implements Drawable // Inheriting Shape and
implementing Drawable
{
   double r; // Radius
```

```
    Circle(String c, double rad) // Child constructor
    {
        super(c); // Calling parent constructor to set color
        r=rad; // Assigning radius
    }

    public void draw() // Overriding interface method
    {
        System.out.println("Drawing a " + color + " Circle.");
        System.out.println("Area = " + (3.14*r*r));
    }
}

class Rectangle extends Shape implements Drawable // Inheriting Shape
and implementing Drawable
{
    double l, b;
    Rectangle(String c, double len, double bre) // Child constructor
    {
        super(c); // Calling parent constructor to set color
        l=len;
        b=bre;
    }

    public void draw() // Overriding interface method
    {
        System.out.println("Drawing a " + color + " Rectangle.");
        System.out.println("Area = " + (l*b));
    }
}

public class ShapeApplication
{
    public static void main(String args[]) // Main Method
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Circle Details: ");
        System.out.print("Colour: ");
        String cc = sc.next();
        System.out.print("Radius: ");
        double cr = sc.nextDouble();

        System.out.println("");
        System.out.println("Enter Rectangle Details: ");
```

```
        System.out.print("Colour: ");
        String rc = sc.next();
        System.out.print("Length: ");
        double rl = sc.nextDouble();
        System.out.print("Breadth: ");
        double rb = sc.nextDouble();

        Circle c = new Circle(cc, cr); // Object creation for Circle
         Rectangle r = new Rectangle(rc, rl, rb); // Object creation for
Rectangle

        System.out.println("");
        c.draw(); // Calling circle method
        System.out.println("");
        r.draw(); // Calling rectangle method

        sc.close();
    }
}
```

ECommerceProductSystem.java

```
// Assignment 7 - Exercise 2: E-Commerce Product System using
Inheritance and Interfaces
import java.util.*;
interface Billable // Interface for product actions
{
    void generateBill(); // Abstract method for billing
}

class Product // Parent class for common properties
{
    String name;
    double price;
    Product(String n, double p) // Constructor
    {
        name=n;
        price=p;
    }
}

class Electronic extends Product implements Billable // Inheriting
Product and implementing Billable
```

```
{
   int warranty; // Specific property for electronics
   Electronic(String n, double p, int w) // Constructor
   {
       super(n,p); // Parent initialization for name and price
       warranty=w;
   }
   public void generateBill() // Implementing interface method
   {
        System.out.println("Electronic: " + name + " | Price: $" + price
+ " | Warranty: " + warranty + " months");
   }
}

class Clothing extends Product implements Billable // Inheriting Product
and implementing Billable
{
   String size;
   Clothing(String n, double p, String s) // Constructor
   {
       super(n,p); // Parent initialization for name and price
       size=s;
   }
   public void generateBill() // Implementing interface method
   {
        System.out.println("Clothing: " + name + " | Price: $" + price +
" | Size: " + size);
   }
}

class Grocery extends Product implements Billable // Inheriting Product
and implementing Billable
{
   String expiry;

   Grocery(String n, double p, String e) // Constructor
   {
       super(n, p); // Parent initialization for name and price
       expiry = e;
   }
   public void generateBill() // Implementing interface method
   {
        System.out.println("Grocery: " + name + " | Price: $" + price + "
| Expiry: " + expiry);
```

```
    }
}

public class ECommerceProductSystem
{
    public static void main(String args[]) // Main method
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Laptop Details: ");
        System.out.print("Name: ");
        String en = sc.nextLine();
        System.out.print("Price: ");
        double ep = sc.nextDouble();
        System.out.print("Warranty (Months): ");
        int ew = sc.nextInt();

        System.out.println("");
        System.out.println("Enter T-Shirt Details: ");
        System.out.print("Name: ");
        String cn = sc.next();
        System.out.print("Price: ");
        double cp = sc.nextDouble();
        System.out.print("Size: ");
        String cs = sc.next();

        System.out.println("");
        System.out.println("Enter Milk Details: ");
        System.out.print("Brand: ");
        String gn = sc.next();
        System.out.print("Price: ");
        double gp = sc.nextDouble();
        System.out.print("Expiry Date: ");
        String ge = sc.next();

         Electronic e = new Electronic(en, ep, ew); // Electronic object
creation
           Clothing c = new Clothing(cn, cp, cs); // Clothing object
creation
        Grocery g = new Grocery(gn, gp, ge); // Grocery object creation

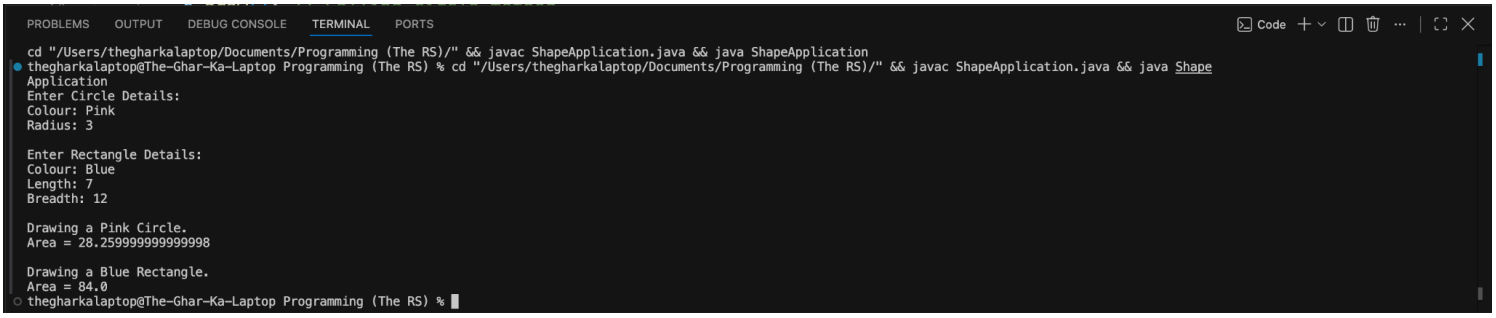
        System.out.println("");
        System.out.println("\n- E-Commerce Bill -");
        e.generateBill(); // Calling electronic billing
```

```
        c.generateBill(); // Calling clothing billing
        g.generateBill(); // Calling grocery billing

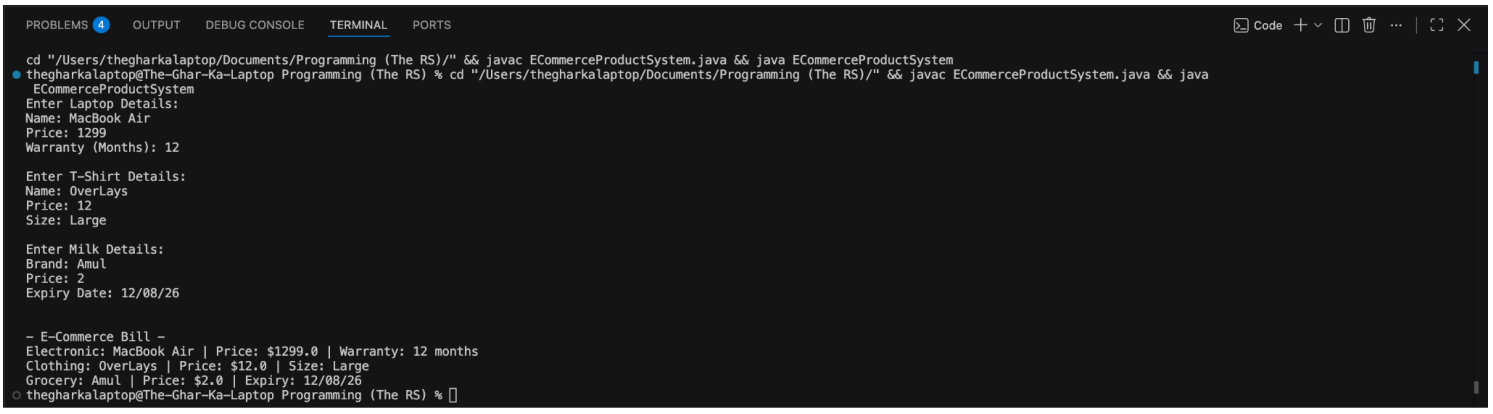
        sc.close();
    }
}
```

Output:

ShapeApplication.java



ECommerceProductSystem.java



Conclusion:

In this assignment, concepts of code reusability and abstraction using inheritance and interfaces were successfully implemented and verified. Inheritance allowed child classes to dynamically acquire common properties from a parent class, significantly reducing code redundancy, while interfaces established a strict contract for required methods across diverse classes.